

Trainer Guide: JDBC - Java Database Connectivity

A slide-by-slide read-aloud script for the jdbc-theory.html deck. Everything on screen, explained in plain spoken language.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: jdbc-theory.html (18 slides)

How to use this guide: each slide below has an ON SCREEN reminder and a SAY section you can read out word-for-word. EMPHASISE and DELIVERY TIP notes are for you only - do not read those aloud.

Slide 1 - Title: JDBC

ON SCREEN

The blue title slide: 'JDBC - Java Database Connectivity', and the line 'the foundation every ORM is built on'.

SAY (read aloud)

Welcome back. Today we tackle JDBC - Java Database Connectivity. This is the piece that finally lets our programs remember things. Up to now, our data has lived in an ArrayList, in memory, and it vanishes the moment the program stops. JDBC is how Java reads from and writes to a real database, so the data survives. And here is why it matters even if you go straight to Spring: JDBC is the foundation that Hibernate and Spring Data JPA quietly run on top of. When you later call repository dot save, JDBC is what actually executes underneath. Learn it today, and all that 'magic' later becomes obvious.

DELIVERY TIP

Set the tone: this session is theory plus short code snippets now, and a full hands-on CRUD lab afterwards. Keep slide 8, the five steps, in their minds for the lab.

Slide 2 - Agenda

ON SCREEN

Seven numbered agenda items, from the persistence problem through architecture, drivers, interfaces, the five steps, SQL injection, and the DAO pattern.

SAY (read aloud)

Here is the plan. We start with the persistence problem - the reason JDBC exists at all. Then the architecture and the role of the driver. We will look at the four driver types, and the four core interfaces you will use in every program. Then the five JDBC steps - the recipe every database program follows. After that, the highlight of the session: SQL injection, and how PreparedStatement defeats it. And we finish with exception handling and the DAO pattern for organising the code cleanly. The theory is front-loaded, because every concept here maps directly to something you will type in the lab.

Slide 3 - The persistence problem

ON SCREEN

Two code panels: an ArrayList with five students while the JVM runs, and the same list empty after a restart.

SAY (read aloud)

Let's open with a question. Your application has an ArrayList with a thousand students added through the day. Then the server restarts - a deploy, a crash, a power blip. Where is your data now? Look at the two panels. On the left, while the program runs, the list has five students - everything is fine. On the right, after a restart, the list is empty. The objects lived in RAM, and RAM is cleared, so they are gone. This is the single idea the whole session rests on: in-memory data lives only as long as the JVM lives. A database persists data beyond the process - it survives restarts, crashes, and redeploys. JDBC is the bridge from your Java objects to that durable store.

Architecture & Interfaces

Slide 4 - What is JDBC?

ON SCREEN

Three cards: 'just interfaces', 'vendor provides the driver', and 'the foundation of ORMs'. A key box about interface-based design.

SAY (read aloud)

So what exactly is JDBC? The crucial insight is that JDBC is mostly a set of interfaces - it lives in the java dot s-q-l package, and it is not a framework, just a set of contracts that say what operations exist. Card two: the database vendor provides the actual implementation, shipped as a JAR file - that JAR is called the driver. One driver per database. Card three: every modern database framework - JPA, Hibernate, Spring Data - sits on top of JDBC and generates these calls for you. Why interfaces? Because your code talks to interfaces, it stays the same whether the database underneath is MySQL, Oracle, PostgreSQL, or Derby. You swap the driver JAR and the connection URL, and your logic is untouched.

EMPHASISE

Drive the punchline home: 'Learn JDBC today, and Spring Data JPA will never feel like magic again.'
This framing keeps even the Spring-focused learners engaged.

Slide 5 - JDBC architecture

ON SCREEN

A vertical layered diagram: Java Application, JDBC API, DriverManager, JDBC Driver, Database - with arrows between each.

SAY (read aloud)

Here is how the pieces fit, top to bottom. At the top is your Java application - your code. Below it, the JDBC API, the java dot s-q-l interfaces like Connection, PreparedStatement, and ResultSet. Below that sits DriverManager. Think of it as a matchmaker: you hand it a connection URL, and it finds and loads the right driver. Next is the JDBC driver itself - the vendor's JAR, the concrete implementation that knows how to speak one specific database's protocol. And at the bottom, the database - MySQL, Derby, H2, whatever. The key thing: your code only ever touches the top two layers. Everything below is handled for you once you have the driver on the classpath.

Slide 6 - The four driver types

ON SCREEN

A four-row table: Type 1 JDBC-ODBC bridge (removed), Type 2 Native API (rare), Type 3 Network protocol (legacy), Type 4 Thin pure-Java (industry standard).

SAY (read aloud)

A little history, because interviewers ask and legacy code exists. JDBC drivers come in four types. Type 1, the JDBC-ODBC bridge, translated calls to ODBC - it was removed back in Java 8. Type 2, the Native API driver, used the database's own C or C-plus-plus libraries and needed a local install - rare today. Type 3, the Network protocol driver, routed calls through a middleware server - you see it in some legacy enterprise systems. And Type 4, the thin or pure-Java driver - a single Java JAR that talks directly to the database's wire protocol. This is the industry standard.

EMPHASISE

Tell them clearly: Type 4 is the only one you will use. It is what you add to pom.xml - one JAR, no local database client needed. MySQL Connector slash J, the Derby driver, the H2 driver - all Type 4.

Slide 7 - The four core interfaces

ON SCREEN

Four cards: DriverManager, Connection, PreparedStatement, ResultSet, each with a one-line role. A note with the chain analogy.

SAY (read aloud)

These four interfaces are the spine of every JDBC program, so let's meet them. DriverManager is a utility class: you give it a URL and credentials, and it returns a Connection. A Connection is one open session with the database. Creating it opens a real network socket, so it is precious - you must always close it. A PreparedStatement is a pre-compiled, parameterised SQL query with question-mark placeholders. It is safe and fast, and you will use it for everything. And a ResultSet is a cursor over the rows that a SELECT returns - you call r-s dot next to walk forward one row at a time. The note ties them together as a chain: DriverManager gets you a Connection; from the connection you create a PreparedStatement; running a SELECT on it gives you a ResultSet to read.

Slide 8 - The five JDBC steps

ON SCREEN

A numbered list, each step with its line of code: register driver, open connection, prepare statement, execute, close.

SAY (read aloud)

This is the recipe behind every JDBC program - keep it in mind for the entire lab. Step one: register the driver, with `Class.forName`. This is automatic in JDBC 4 and later, but we show it so you understand what is happening. Step two: open a connection, with `DriverManager.getConnection`, passing the URL, username, and password. Step three: prepare the statement, with `connection.prepareStatement`, passing your SQL. Step four: execute. Use `executeUpdate` for insert, update, and delete; use `executeQuery` for a `SELECT`, which returns a `ResultSet`. Step five: close, to release the resource - or, better, use `try-with-resources`, which we will cover shortly.

EMPHASISE

Say it plainly: every program you write today - insert, select, update, delete - follows these same five steps. Once they see the shape, the lab becomes pattern-matching.

PreparedStatement & SQL Injection

Slide 9 - Statement vs PreparedStatement

ON SCREEN

Two code panels: Statement built by string concatenation (bad), PreparedStatement with placeholders (good). Three benefit cards.

SAY (read aloud)

There are two ways to send SQL, and only one is acceptable. On the left is Statement, built by gluing strings together with plus signs. It works, but it is recompiled on every call, it is messy with all those quotes, and - most importantly - it is open to injection. On the right is PreparedStatement. You write the SQL once with question-mark placeholders, then bind the values with `setInt`, `setString`, and so on. The parameter index starts at one, not zero - worth flagging. The three cards summarise the wins: performance, because the template is compiled once and reused; readability, because there is no string soup; and security, because parameters are never treated as SQL - which is the next two slides.

Slide 10 - SQL injection: the attack

ON SCREEN

A login query built by concatenation, then the attacker input `admin--`, and the resulting query with the password check commented out. A red danger box.

SAY (read aloud)

This is the most memorable five minutes of the session, so let's slow down. Imagine a login query built the lazy way, by concatenating the username and password into the SQL string. With normal input it works fine. But watch what a malicious user types into the username field: admin, then a single quote, then space dash dash. When that gets glued into the query, the single quote closes the username string early, and the dash dash starts a SQL comment - which comments out the entire password check. The attacker is now logged in as admin with no password at all. And it gets worse. If they type a quote, semicolon, DROP TABLE users, dash dash - they can delete your whole table.

EMPHASISE

Make it real: SQL injection has been in the OWASP Top 10 most critical web vulnerabilities for over a decade. This is how real companies get breached - and one line of PreparedStatement prevents all of it.

Slide 11 - SQL injection: the defense

ON SCREEN

Side-by-side: the vulnerable Statement login and the safe PreparedStatement login. A note explaining how setString escapes input. A key box with the rule.

SAY (read aloud)

The fix is almost embarrassingly simple. On the right, the same login, but with a PreparedStatement and question-mark placeholders. We bind the username and password with setString. Here is why that defeats the attack: the driver sends the SQL template and the values separately. The values are never parsed as SQL - they are bound as literal data. So the attacker's admin-quote-dash-dash is treated as a username that literally contains those characters. The database looks for a user with that exact strange name, finds none, and the login fails. The quote can no longer escape out of the value. The rule, in the key box, is absolute: never use Statement for any query that includes user input. Always use PreparedStatement. No exceptions.

Slide 12 - executeQuery vs executeUpdate

ON SCREEN

A two-row table: executeQuery for SELECT returns a ResultSet; executeUpdate for INSERT/UPDATE/DELETE/CREATE returns an int row count. A warning box.

SAY (read aloud)

A small but important distinction that trips up beginners constantly. There are two execute methods. `executeQuery` is for `SELECT` - reading data - and it returns a `ResultSet`, the rows. `executeUpdate` is for `INSERT`, `UPDATE`, `DELETE`, and `CREATE` - anything that changes data or schema - and it returns an `int`: the number of rows affected. Read the warning: if you use `executeUpdate` for a `SELECT`, you get back zero, not your data. If you use `executeQuery` for an `INSERT`, it throws an exception. Match the method to the SQL verb. And there is a bonus in that row count - it tells you whether anything actually changed. An `UPDATE` on an id that does not exist returns zero, which is how you tell 'ran fine but matched nothing' from 'found it and changed it'.

Robust Code & the Big Picture

Slide 13 - Exception handling: `SQLException`

ON SCREEN

A try-catch printing three things, and three cards: `getMessage`, `getSQLState`, `getErrorCode`. A note listing a few `SQLState` codes.

SAY (read aloud)

JDBC methods throw `SQLException`, which is a checked exception. Do not just swallow it with a generic `catch-Exception`, because `SQLException` carries three pieces of diagnostic gold. `getMessage` gives a human-readable description. `getSQLState` gives a standardised five-character code that is portable across databases. And `getErrorCode` gives a vendor-specific integer for the exact error. Always log all three. The note lists a few you will actually meet: 42-X-05 means table not found; 23505 means duplicate primary key; 08001 means it could not connect to the database. This is precisely the information your DevOps team needs when diagnosing a production database issue, so get into the habit now.

Slide 14 - try-with-resources

ON SCREEN

Two panels: the old verbose `finally`-block that is easy to get wrong, and the clean `try-with-resources` that auto-closes. A key box with the rule.

SAY (read aloud)

A `Connection` is a real network socket. If an exception fires before you close it, the connection leaks - and enough leaks will exhaust the database. On the left is the old way: a `finally` block with null checks and a nested `try` just to close. Fifteen-plus lines, and easy to forget. On the right is `try-with-resources`. You declare your `Connection` and `PreparedStatement` inside the `try` parentheses, and Java closes them automatically - on success and on exception, in reverse order. Clean and safe. The rule, in the key box: `Connection`, `Statement`, `PreparedStatement`, and `ResultSet` all implement `AutoCloseable`. Always use `try-with-resources` in production code.

Slide 15 - The DAO pattern

ON SCREEN

Three cards - DBConnector, DAO, Tester - and a flow diagram from Tester to DAO to DBConnector to Database. A key box linking to JpaRepository.

SAY (read aloud)

Mixing connection code, SQL, and Scanner menus in one class becomes a tangle fast. The DAO pattern - Data Access Object - splits the work into three clean layers. DBConnector is one class with one job: a getConnection method. No SQL, no UI. The DAO holds all the SQL operations as methods - insert, getAll, getByld, update, delete. No UI code here. And the Tester, or main class, has the Scanner menu, calls the DAO methods, and displays results. No SQL here. Why does this pay off? If you switch from Derby to MySQL, you change only the DBConnector's URL and driver - the business logic and the UI are untouched.

EMPHASISE

Land the connection to what they already know: Spring Data's JpaRepository IS a DAO. When you call repository dot findByld, it runs the exact JDBC you are about to write in the lab - just generated for you automatically.

Slide 16 - JDBC vs Spring Data JPA

ON SCREEN

A comparison table across code volume, control, learning curve, and best-for. A key box with the takeaway.

SAY (read aloud)

Let's place JDBC in the bigger picture honestly. Code volume: raw JDBC is high - you write the SQL and map the ResultSet by hand. Spring Data JPA is low - often just a method name or an annotation. Control: JDBC gives you full control to write any SQL you want. JPA gives moderate control, with JPQL and the at-Query annotation for custom cases. Learning curve: JDBC is low - just SQL and four interfaces. JPA is higher, because you need JPA concepts first. Best for: JDBC suits legacy systems and complex hand-tuned SQL; JPA suits most modern applications. The takeaway: you learn JDBC to understand the foundation, and you use Spring Data JPA to build efficiently on top of it.

Recap & Close

Slide 17 - Session recap

ON SCREEN

A checklist of six things they should now be able to explain and do.

SAY (read aloud)

Let's recap the six things you should now own. One: the JDBC architecture, and that the driver is a vendor-supplied JAR implementing Java's interfaces. Two: the four driver types, and that Type 4, the pure-Java thin driver, is today's standard. Three: the four core interfaces - DriverManager, Connection, PreparedStatement, and ResultSet - and what each does. Four: the five JDBC steps - register, connect, prepare, execute, close. Five: why PreparedStatement always beats Statement - performance, readability, and injection safety. And six: try-with-resources for safe resource management, and the DAO pattern for clean separation of concerns.

Slide 18 - Closing

ON SCREEN

The closing blue slide: 'Let's build it.'

SAY (read aloud)

And that wraps the theory. The one sentence to carry forward: everything Spring Data JPA does is built on top of exactly this. When you call repository dot save, JDBC is running underneath - and now you know what it is doing. Now let's open the lab sheet. We will connect to Derby, build the employees table, and write full CRUD with PreparedStatement, then refactor it into a clean DAO. Let's build it.

DELIVERY TIP

Transition straight into the hands-on lab. Have the Derby connection ready on your machine so the first 'Connection Established' moment works instantly and builds momentum.